

Final Exam — Sample Solutions

Software Engineering — UATX — Spring 2026

Question 1

T1 — JavaScript & TypeScript (5.1) · Trace-through

The script runs to completion. Give the value printed at A, B, C, and D.

```
const todos = [
  { id: 1, title: "read notes", done: true, priority: 2 },
  { id: 2, title: "write fetch", done: false, priority: 1 },
  { id: 3, title: "pick up milk", done: false, priority: 3 },
  { id: 4, title: "call the vet", done: true, priority: 1 },
];

const a = todos.filter(t => !t.done).map(t => t.title);
console.log("A:", a); // ____

const b = todos.filter(t => t.done).length;
console.log("B:", b); // ____

const c = todos.filter(t => !t.done).reduce((s, t) => s + t.priority, 0);
console.log("C:", c); // ____

console.log("D:", 1 === "1", 0 == false, null === undefined); // ____
```

SOLUTION

A: ["write fetch", "pick up milk"]. B: 2. C: 4 (priorities 1 + 3 of the two pending todos).

D: false true false. === does no coercion so 1 === "1" is false; == coerces, so 0 == false is true; null === undefined is false (only loose == makes those two equal).

Question 2

A promo code grants \$10 of account credit, once per user. On a flaky connection a user double-taps redeem, so two identical requests arrive at nearly the same instant. The handler is below; both requests run their `SELECT` before either runs its `INSERT`.

```
def redeem(user_id, code):
    already = db.execute(
        text("SELECT 1 FROM redemptions WHERE user_id = :u AND code = :c"),
        {"u": user_id, "c": code},
    ).first()
    if already:
        raise HTTPException(409, "already redeemed")
    db.execute(text("INSERT INTO redemptions (user_id, code) VALUES (:u, :c)"),
               {"u": user_id, "c": code})
    db.execute(text("UPDATE accounts SET credit = credit + 10 WHERE user_id = :u"),
               {"u": user_id})
```

1. What happens, and how much credit does the user end up with?
2. A teammate wraps the whole handler in one transaction (with `db.engine.begin()` as `conn:` for each request). Does that fix the issue? Explain.
3. What actually prevents it? Give one concrete mechanism and why it is atomic.

SOLUTION

1. Both `SELECT`s see no prior redemption, both pass the 409 guard, and both run the `INSERT` and the +\$10 update. The user ends with \$20 of credit (and two redemption rows) for a once-only code.
2. No. Wrapping each request in a transaction makes that request's writes commit or roll back together, but it does not lock the rows the `SELECT` read, so both requests still run their `SELECT`, see nothing, and proceed. A transaction is not a lock; on its own it does not serialize the two requests.
3. A `UNIQUE(user_id, code)` constraint on `redemptions` makes the second `INSERT` fail atomically (`IntegrityError -> 409`) regardless of timing; or take a row lock (`SELECT ... FOR UPDATE`) so the second request waits for the first to commit. The database enforces it atomically, unlike the app-level check-then-act.

Question 3

T2 — Async, fetch & races (5.2) · Spot the bug

Your agent produced this submit handler to POST a new message and show it. It works when you click slowly on a good connection. Find at least three defects; for each, one sentence on what goes wrong and one on the fix.

```
const form = document.querySelector("#new") as HTMLFormElement;
const input = document.querySelector("#text") as HTMLInputElement;
const list = document.querySelector("#list") as HTMLUListElement;

form.addEventListener("submit", async (event) => {
  const response = await fetch("/api/messages", {
    method: "POST",
    body: { text: input.value },
  });
  const created = await response.json();
  const li = document.createElement("li");
  li.textContent = input.value;
  list.append(li);
});
```

SOLUTION

1. No `event.preventDefault()`. The form submits and reloads the page, discarding the async work. Call it first.
2. `body` is a raw object. It must be `JSON.stringify({ text: input.value })` plus a `headers: { "Content-Type": "application/json" }`, or the server cannot read it.
3. No `response.ok` check. `fetch` does not throw on 4xx/5xx, so a rejected POST still appends an item. Guard with `if (!response.ok) throw ...` inside a try/catch.

Bonus: it renders `input.value` instead of `created.text`, so it shows the raw input rather than the server's stored version (id, trimming, normalization).

Question 4

Your agent produced this endpoint for a project-management app (`TaskUpdate` is a Pydantic model). It parameterizes its values and it checks authorization. Read it carefully.

```
@app.patch("/api/projects/{project_id}/tasks/{task_id}")
def update_task(project_id: int, task_id: int, body: TaskUpdate,
                user: User = Depends(current_user)):
    with engine.begin() as conn:
        member = conn.execute(
            text("SELECT 1 FROM project_members "
                "WHERE project_id = :p AND user_id = :u"),
            {"p": project_id, "u": user.id},
        ).first()
        if member is None:
            raise HTTPException(403, "not a member of this project")

        fields = body.model_dump(exclude_unset=True)
        if not fields:
            raise HTTPException(400, "nothing to update")
        set_clause = ", ".join(f"{col} = :{col}" for col in fields)
        result = conn.execute(
            text(f"UPDATE tasks SET {set_clause} WHERE id = :id"),
            {**fields, "id": task_id},
        )
        if result.rowcount == 0:
            raise HTTPException(404)
        return {"ok": True}
```

1. Name two things this endpoint does well, security-wise.
2. It has a serious access-control hole. Describe it, give a concrete request that exploits it, and give the fix.
3. The `SET` clause is built from whatever fields are in the request body. Explain the risk that creates, and what `TaskUpdate` has to guarantee for it to be safe.

SOLUTION

1. Any two: it checks the caller is a member of the project before writing anything (a real authorization gate, not just authentication); it binds its values as parameters, so the inputs cannot inject SQL; it does the read-and-write in one transaction; it 400s on an empty update and 404s when no row matches.
2. Container-vs-item gap: the membership check confirms the caller is on `project_id`, but the `UPDATE` matches `WHERE id = :id` only -- nothing ties the task to that project. A member of project 5 sends `PATCH /api/projects/5/tasks/9999` where task 9999 belongs to project 200; membership passes and the write lands on another project's task. Fix: scope the write to the container -- ... `WHERE id = :id AND project_id = :p` (rowcount 0 -> 404).
3. The columns written are whatever keys the body carries, so the endpoint is only as safe as `TaskUpdate`. If it strictly whitelists the user-editable fields (title, description, status), fine; if it also

accepts something like `project_id` or `owner_id`, a user can move the task or change fields they should not (mass assignment); if it is a loose/ dict model, the column name in `{col} = :{col}` is attacker-controlled and becomes SQL injection. `TaskUpdate` must be a small, purpose-built whitelist of exactly the columns a member may edit.

Question 5

T3 — React: components & rendering (6.1) · Specification

Feature request to your agent: "As the user types in the search box, show matching products." The agent's first cut is below, and it works in a quick demo. Before you let it ship, write the spec you would hand back. Do not write code.

```
function ProductSearch() {
  const [results, setResults] = useState([]);
  const [query, setQuery] = useState("");
  useEffect(() => {
    fetch(`/api/products?q=${query}`)
      .then(r => r.json())
      .then(setResults);
  }, [query]);
  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} />
      <ul>{results.map(p => <li key={p.id}>{p.name} (${p.price})}</li>)}</ul>
    </div>
  );
}
```

1. The display states this UI must handle -- name them.
2. Two behaviors it is currently missing or getting wrong (look hard at a fast typist on a flaky network).
3. One product decision that is ambiguous, and the call you would make.

SOLUTION

1. Loading, error, empty ("no matches"), and results -- plus the initial state, where an empty query currently fires a fetch for everything.
2. Any two: no `response.ok` / error handling, so a failed request silently shows nothing; out-of-order responses (the 5.2 race) -- type "cat" fast and the slower "ca" response can land last and overwrite the newer results, so superseded requests should be cancelled/ignored (`AbortController`); a fetch on every keystroke hammering the API (wants `debounce`); an empty query probably should not fetch at all.
3. A real ambiguity, e.g.: does an empty box show all products, nothing, or recent/popular? Is there a minimum query length? State your call.

Question 6

This workflow runs the test suite on every push; two of the tests are shown below. On the developer's laptop the suite passes every time. In CI, `test_lists_single_user` fails about half the time.

```
# .github/workflows/test.yml
name: Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with: { python-version: "3.14" }
      - run: pip install -r requirements.txt
      - run: pytest -n auto          # runs tests in parallel across CPUs
```

```
# conftest.py + test_users.py (app talks to the module-level db.engine)
@pytest.fixture
def client():
    return TestClient(app)

def test_create_user(client):
    r = client.post("/users", json={"username": "alice"})
    assert r.status_code == 201

def test_lists_single_user(client):
    users = client.get("/users").json()
    assert len(users) == 1
```

1. Why does `test_lists_single_user` pass locally but fail intermittently in CI?
2. Is the bug in the workflow or in the tests? Point to the specific cause.
3. Give two fixes -- one to the tests, one you could make to the workflow -- and say which is the real fix.

SOLUTION

1. The tests share one database (the module-level `db.engine`) with no per-test isolation, and `test_lists_single_user` asserts an exact global count. Locally the suite runs serially in a stable order against a known DB; CI runs on a fresh VM with `pytest -n auto`, so `test_create_user` (and any other user-creating test) runs in parallel and the count is 0, 1, or 2 unpredictably.
2. The bug is in the tests: no isolation, plus an assertion on global state (`len(users) == 1`) that only holds under a specific order/state. The workflow merely exposes it via the fresh VM and parallelism.
3. Test fix (the real one): isolate each test with a `test_db` fixture (swap `db.engine` to a fresh in-memory engine per test) and have the test arrange its own data and assert on that user, not a global

count. Workflow fix (a band-aid): drop `-n auto` so tests run serially -- it hides the flake but leaves the shared-state bug.

Question 7

T6 — Docker (8.1) · Short design

Your agent generated this Dockerfile for the FastAPI service. It is the team's deploy image: pushed to a shared registry (GitHub Container Registry) that teammates and the deploy server pull from, and rebuilt on every push to `main`. The project directory also contains a `.env` with more secrets, and there is no `.dockerignore`.

```
FROM python:3.14
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# image-processing system library the app shells out to
RUN apt-get update && apt-get install -y imagemagick

ENV JWT_SECRET="prod-signing-key-7f3a9c2b"
ENV DATABASE_URL="postgresql://app:s3cr3t@db.internal:5432/app"

CMD uvicorn app:app --host 0.0.0.0 --port $PORT
```

1. The team rebuilds this image many times a day as they edit application code, and rebuilds are slower than they should be. One thing here is already done right and one step is the culprit -- identify both, and restructure so a one-line change to `app.py` rebuilds quickly.
2. This image is pushed to a shared registry. What secrets does it expose, by what mechanisms, and how do you keep them out of the image?
3. You realize this image was already pushed last week. You delete the secrets from the Dockerfile and add a `.dockerignore` now. Is that enough? What do you actually do?

SOLUTION

1. Already right: `requirements.txt` is copied and installed before `COPY . .`, so the dependency layer stays cached when only code changes -- do not "fix" that. The culprit: the `apt-get install imagemagick` step sits AFTER `COPY . .`, so a slow system install re-runs on every code edit (and so does everything below it). Restructure so stable, slow steps are high and fast-changing code is last: move the `apt-get install` near the top, then `COPY requirements.txt + pip install`, then `COPY . .`. Now a one-line `app.py` change invalidates only the final copy layer.
2. Two vectors. (a) `ENV JWT_SECRET=...` / `ENV DATABASE_URL=...` hardcode real secrets into the image (readable via `docker history` and to anyone who pulls) and into the Dockerfile, which lives in the repo. (b) With no `.dockerignore`, `COPY . .` also bakes the project's `.env` into a layer. Fix: take secrets out of the Dockerfile entirely and inject them at run time (`-e` / `--env-file` / dashboard), and add a `.dockerignore` (excluding `.env`, `.git`, `__pycache__`) so nothing secret is copied in.
3. Not enough. The secrets are already inside the image that was pushed (its layers and history), so cleaning the Dockerfile now does not change what is already published. Treat them as leaked: rotate

them (new DB password / JWT signing secret), rebuild and repush a clean image, and update the runtime config with the new values.